



D Y PATIL
INTERNATIONAL
UNIVERSITY
AKURDI PUNE

Name : Shweta shelke

PRN: 20220802278

LAB-04

Introduction

This project implements and compares the performance of different consensus algorithms—Proof of Work (PoW), Proof of Stake (PoS), and Practical Byzantine Fault Tolerance (pBFT)—on a simulated blockchain network. The goal is to analyze their efficiency, security, and computational requirements.

Objectives

- Implement PoW, PoS, and pBFT consensus mechanisms.
- Simulate a blockchain network to test these algorithms.
- Measure and compare performance metrics such as transaction throughput, latency, energy consumption, and security.

Proof of Work (PoW)

- Requires miners to solve computational puzzles to add a block.
- High energy consumption but ensures security through cryptographic hashing. •
Used in Bitcoin.

Code of POW:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract ProofOfWork {
5
6     uint public difficulty; // Difficulty level for POW
7     uint public reward;     // Reward for solving the POW
8     uint public blockCount; // Counter for the number of blocks added
9     mapping(address => uint) public balances; // User balances
10
11     event BlockMined(address miner, uint nonce, bytes32 blockHash);
12
13     // Constructor to set the initial difficulty and reward
14     constructor(uint _difficulty, uint _reward) {
15         difficulty = _difficulty;
16         reward = _reward;
17         blockCount = 0;
18     }
19
20     // Function to simulate mining process
21     function mine(uint nonce) public {
22         // Create a hash by combining the nonce, current block timestamp, and blockCount
23         bytes32 blockHash = keccak256(abi.encodePacked(nonce, block.timestamp, blockCount));
24
25         // Calculate the target threshold based on difficulty
26         uint target = 2**(256 - difficulty);
27
28         // Check if the hash is less than the target (valid POW solution)
29         if (uint(blockHash) < target) {
30             // If valid, increment block count, reward the miner, and emit event
31             blockCount++;
32             balances[msg.sender] += reward; // Miner gets rewarded
33             emit BlockMined(msg.sender, nonce, blockHash);
34         } else {
35             revert("Invalid POW solution! Try again.");
36         }
37     }
38
39     // View function to check the current block count (simulating blockchain length)
40     function getBlockCount() public view returns (uint) {
41         return blockCount;
42     }
43
44     // View function to check the balance of a miner
45     function getBalance() public view returns (uint) {
46         return balances[msg.sender];
47     }
48 }

```

Output of POW:

[illegible]

Proof of Stake (PoS)

- Selects validators based on their stake in the network.
- More energy-efficient than PoW while maintaining security.

- Used in Ethereum 2.0.

Code of POS:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract ProofOfStake {
5
6     mapping(address => uint) public stakes;
7     mapping(address => uint) public balances;
8
9     constructor() {} 288322 gas 288000 gas
10
11     // Stake tokens to participate in the consensus
12     function stake(uint amount) public { infinite gas
13         require(amount > 0, "Amount should be greater than 0");
14         stakes[msg.sender] += amount;
15     }
16
17     // Simulate the selection of a validator based on stake
18     function validate(uint amount) public returns (bool) { infinite gas
19         require(stakes[msg.sender] >= amount, "Insufficient stake");
20
21         // Simple simulation of staking rewards: if staked amount is sufficient, reward
22         uint reward = amount * 10; // For simplicity, 10 times the staked amount is rewarded
23         balances[msg.sender] += reward;
24         return true;
25     }
26 }
27

```

Output of POS

[illegible]

Practical Byzantine Fault Tolerance (pBFT)

- A leader-based consensus that works efficiently in permissioned blockchain networks.
- Faster than PoW and PoS but requires a limited number of nodes for practical use.
- Used in Hyperledger Fabric.

Code of Pbft:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract PracticalBFT {
5
6     address[] public validators; // List of validator addresses
7     mapping(address => bool) public hasVoted; // Tracking if a validator has already voted for a proposal
8     uint public validatorCount; // Number of validators
9     uint public requiredVotes; // 2/3 majority required for consensus
10    uint public blockCount; // Simulating blockchain length
11    mapping(uint => uint) public votesForTransaction; // Transaction ID => vote count
12
13    event TransactionValidated(uint transactionID, address validator);
14    event BlockAdded(uint blockID, address validator);
15
16    constructor(address[] memory _validators) {
17        validators = _validators;
18        validatorCount = _validators.length;
19        requiredVotes = (validatorCount * 2) / 3 + 1; // 2/3 majority needed
20        blockCount = 0;
21    }
22
23    // Function for validators to vote on a transaction
24    function voteOnTransaction(uint transactionID) public {
25        require(isValidator(msg.sender), "You are not a validator");
26        require(!hasVoted[msg.sender], "You have already voted");
27
28        // Record that this validator has voted
29        hasVoted[msg.sender] = true;
30        votesForTransaction[transactionID]++;
31
32        emit TransactionValidated(transactionID, msg.sender);
33
34        // If enough votes have been gathered, validate the transaction
35        if (votesForTransaction[transactionID] >= requiredVotes) {
36            addBlock();
37        }
38    }
39
40    // Internal function to simulate adding a new block when the transaction is validated
41    function addBlock() internal {
42        blockCount++; // simulate adding a new block to the blockchain
43        resetVotes(); // Reset all votes after adding the block
44
45        emit BlockAdded(blockCount, msg.sender); // Emit event of new block being added
46    }

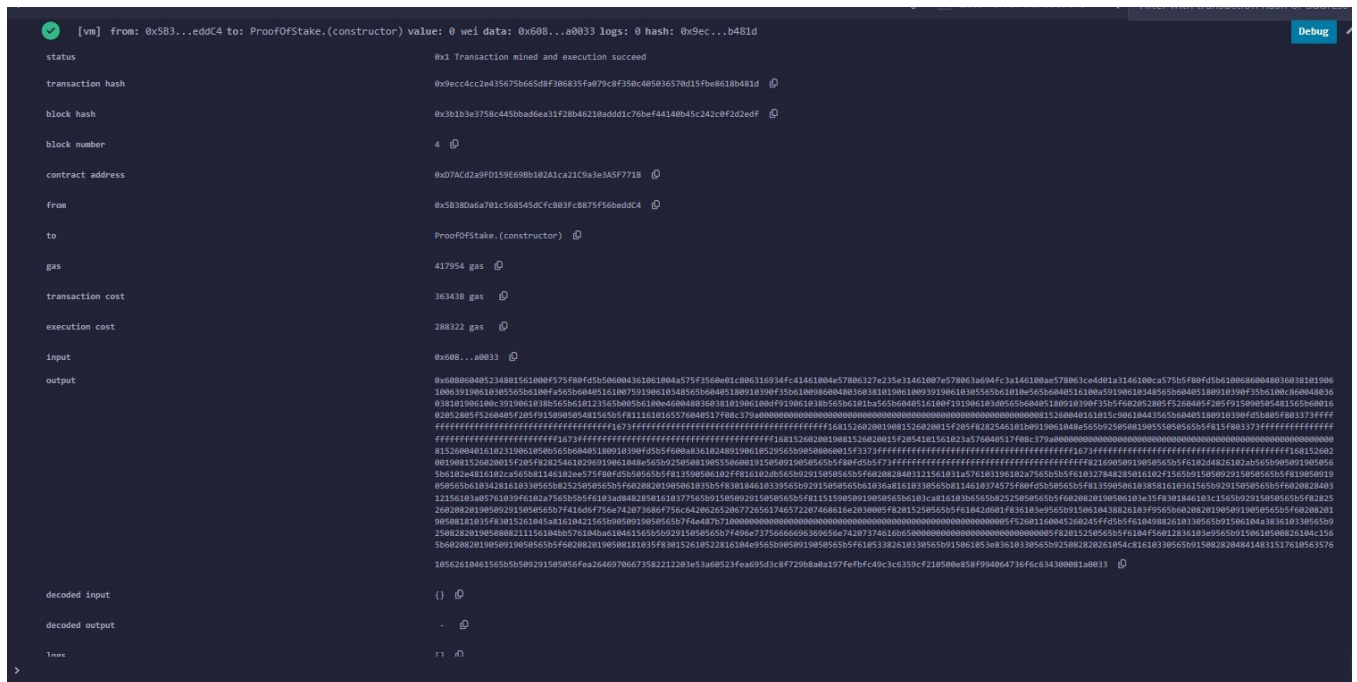
```

```

45        emit BlockAdded(blockCount, msg.sender); // Emit event of new block being added
46    }
47
48    // Reset all votes after a block is added
49    function resetVotes() internal {
50        for (uint i = 0; i < validators.length; i++) {
51            hasVoted[validators[i]] = false;
52        }
53    }
54
55    // Helper function to check if an address is a validator
56    function isValidator(address _address) internal view returns (bool) {
57        for (uint i = 0; i < validators.length; i++) {
58            if (validators[i] == _address) {
59                return true;
60            }
61        }
62        return false;
63    }
64
65    // View function to get the number of blocks in the "blockchain"
66    function getBlockCount() public view returns (uint) {
67        return blockCount;
68    }
69
70    // View function to check if a validator has voted on a specific transaction
71    function hasValidatorVoted(address validator) public view returns (bool) {
72        return hasVoted[validator];
73    }
74 }
75

```

Output of Pbft:



Conclusion

The comparison of PoW, PoS, and pBFT reveals significant differences in their efficiency, security, and suitability for various blockchain applications:

- **PoW** provides strong security through cryptographic puzzles but suffers from high energy consumption and low transaction throughput. It is best suited for decentralized, permissionless blockchains where security is the top priority.
- **PoS** improves energy efficiency and transaction speed while maintaining security. However, it may lead to centralization risks if a few entities hold a large stake. It is ideal for scalable and eco-friendly blockchain networks.
- **pBFT** delivers the fastest transaction processing with low latency and energy consumption but works best in permissioned networks with a limited number of nodes. It is well-suited for enterprise blockchain solutions.

Overall, the choice of a consensus algorithm depends on the specific use case—PoW for robust security, PoS for energy-efficient public blockchains, and pBFT for fast, permissioned networks. Future research can focus on hybrid models that combine the strengths of multiple consensus mechanisms.